

基于ESP32-S3+FreeRTOS的智慧蜂箱终端系统--介绍与问答

一、项目介绍

面试官：你介绍一下这个智慧蜂箱项目，它主要是做什么的？

这个项目来源于全国大学生物联网设计竞赛，主要是想利用物联网和视觉识别技术，解决传统养蜂过程中依赖人工巡检、蜂箱状态不透明以及异常情况难以及时发现的问题。

在做项目之前，我们实际去养蜂场做过调研，也采访了蜂农。我们发现，蜂农通常需要定期打开蜂箱，根据经验判断蜂群状态和蜂蜜产量。蜂箱数量多了以后，人工巡检的工作量比较大，而且频繁开箱也会干扰蜂群。另外，蜂箱被盗和胡蜂入侵也是蜂农比较关注的问题。因此，我们设计了一套“云养蜂”智慧蜂箱系统，希望让蜂农能够远程了解蜂箱状态，并及时发现异常。

整个系统可以分成蜂箱端、云端、后端和应用端几个部分。

蜂箱端使用ESP32-S3作为主控，通过温湿度传感器、称重传感器和GPS模块采集蜂箱内部环境、重量和位置信息，并通过OLED进行本地显示。采集到的数据通过MQTT上传到阿里云物联网平台，再由后端进行存储，最终展示在微信小程序和数据大屏上。这样，用户就可以远程查看蜂箱的环境、重量和位置变化。

除此之外，我们还在蜂箱入口安装了ESP32-CAM，由它提供实时图像流，PC端获取图像后，通过YOLOv8识别胡蜂入侵。这里我们把数据通信分成了两条链路：温湿度、重量和设备状态等轻量数据使用MQTT传输；图像数据量比较大，所以通过HTTP单独传输，避免图像占用MQTT链路，影响传感器数据和控制指令的实时性。

我在团队中主要负责嵌入式终端部分，包括硬件选型、接线和原型组装，以及ESP32-S3和ESP32-CAM两端的程序开发。我还负责阿里云设备注册、物模型和通信配置，并完成嵌入式终端与后端、YOLOv8识别模块之间的接口联调。后端、YOLOv8模型、小程序和数据大屏主要由其他成员负责。

在软件架构上，我使用FreeRTOS将传感器采集、GPS解析、本地显示、网络管理和数据上传拆分成独立任务，并根据任务特点分配到ESP32-S3的两个核心上。任务之间通过Queue、EventGroup和Mutex完成数据传递、网络状态同步和共享资源保护。为了提高终端在网络不稳定场景下的可维护性和稳定性，我还设计了OTA升级、断网数据缓存、恢复后的限速补发以及参数持久化等功能。

最终，我们完成了可以实际运行的智慧蜂箱原型，打通了从蜂箱端采集、设备上云、后端存储到小程序和数据大屏展示的完整链路，也在真实蜂场中测试了温湿度、重量、物联网通信和胡蜂识别功能。这个项目最后获得了全国大学生物联网设计竞赛全国一等奖。

二、面试问答

（一）项目高频追问题库

一、项目与个人职责

1. 你在这个项目中主要负责什么？

我主要负责嵌入式终端部分。硬件方面，包括器件选型、接线和原型组装；软件方面，我编写了ESP32-S3和ESP32-CAM两端的程序，实现了传感器采集、GPS解析、本地显示、MQTT上报和HTTP图传。我还负责阿里云设备注册、物模型和通信配置，以及嵌入式终端与后端、YOLOv8模块之间的接口联调。

后端、MySQL数据库、YOLOv8模型、小程序和数据大屏主要由其他成员负责，我负责为这些模块提供数据或图像接口，并配合完成联调。

2. 整个系统的数据是怎么流转的？

系统主要有两条数据链路。

第一条是传感器数据链路。ESP32-S3采集温湿度、重量和GPS数据，通过MQTT上传到阿里云物联网平台，后端订阅并存入MySQL，最后由小程序和数据大屏进行展示。

第二条是图像链路。ESP32-CAM采集蜂箱入口图像，通过HTTP提供JPEG图片或MJPEG视频流，PC端获取图像后交给YOLOv8进行胡蜂识别。

简单来说，就是一条链路负责蜂箱状态数据，另一条链路负责图像识别。

3. 这个项目中你认为最难的地方是什么？

从我负责的部分来看，难点不是让某一个传感器读出数据，而是让采集、显示、GPS解析、网络通信和图像传输同时运行，并且在网络不稳定时不会互相阻塞。

因此，我后面重点做了三件事：第一，用FreeRTOS把不同功能拆成独立任务；第二，用Queue、EventGroup和Mutex处理任务间的数据传递和资源竞争；第三，增加断网缓存、自动重连和OTA等机制，提高系统的稳定性和可维护性。

二、硬件设计

4. 为什么选择ESP32-S3作为主控？

主要考虑了三个方面。

第一，ESP32-S3自带Wi-Fi，能够直接连接阿里云，不需要额外增加通信模块。第二，它是双核处理器，片上资源也比较充足，适合同时运行传感器采集、GPS解析、本地显示和网络通信任务。第三，它对FreeRTOS、硬件UART、I²C和GPIO等外设支持比较完善，软件生态也比较成熟，适合快速完成竞赛原型。

如果后续做产品化，还需要进一步评估功耗、工业温度范围、防护和长期供电方案。

5. 为什么使用ESP32-S3和ESP32-CAM两个控制器，不能放在一个芯片上吗？

理论上可以尝试放在同一个控制器上，但摄像头采集、JPEG缓冲和HTTP图传会占用较多内存、网络带宽和处理时间，容易影响传感器数据上报和控制指令。

因此，我们把ESP32-S3作为主节点，负责传感器采集和物联网通信；把ESP32-CAM作为独立图像节点，负责蜂箱入口的图像采集和HTTP输出。这样既能隔离两类业务的资源，也便于根据蜂箱结构分别安装。

两个节点之间没有必须存在的有线通信，数据最终在上层系统中汇合。

6. 这些传感器分别是怎么连接的？

DHT22通过厂商定义的单总线时序与ESP32-S3通信；HX711用于读取称重传感器的电桥信号，通过DATA和SCK两根GPIO进行通信；GPS模块通过硬件UART发送NMEA数据；OLED通过I²C连接。

ESP32-CAM连接OV2640摄像头，图像通过芯片内部的I²S和DMA采集，并缓存在PSRAM中。

三、FreeRTOS与并发设计

7. 为什么要使用FreeRTOS，直接写一个大循环不行吗？

功能比较少时，大循环是可以工作的。但这个项目同时包含传感器采集、GPS持续解析、OLED刷新、网络重连、MQTT上传和HTTP服务。

如果全部放在一个循环里，某个网络操作或传感器操作一旦耗时，后面的功能就会一起延迟。例如GPS数据如果不能及时读取，串口缓冲区可能溢出；网络重连耗时过长，也可能导致显示和数据采集停顿。

所以我把不同功能拆成独立任务，通过调度器控制运行，让网络异常尽量不要影响本地采集和显示。

8. 你的FreeRTOS任务是怎么划分的？

我一共创建了六个主要业务任务。

网络通信侧包括网络管理任务、MQTT上传任务和HTTP服务任务；采集与本地业务侧包括传感器采集任务、GPS解析任务和OLED显示任务。

网络相关任务主要绑定在一个核心，传感器采集、GPS和显示任务放在另一个核心。这样划分主要是为了隔离网络波动与本地实时业务，并减少不同类型任务之间的干扰。

这里的六个任务是我创建的业务任务，系统内部还会运行FreeRTOS空闲任务、Wi-Fi和TCP/IP协议栈等系统任务。

9. 任务优先级是怎么设计的？

网络管理任务需要维护Wi-Fi和MQTT状态，所以优先级相对较高；传感器采集和GPS解析直接影响数据完整性，优先级也比较高；MQTT上传处于中间优先级；OLED只负责给用户查看最新状态，即使晚刷新几十毫秒影响也不大，所以优先级最低。

我的原则不是网络任务越高越好，而是根据任务的时效性和阻塞风险设置优先级，并确保高优先级任务能够及时阻塞或让出CPU。

10. Queue、EventGroup和Mutex分别用来做什么？

Queue用于传递完整的采集数据。传感器任务只负责采集，然后把数据放入队列；上传任务从队列中取数据并发送，这样采集和网络上传不会直接耦合。

EventGroup用于保存Wi-Fi和MQTT连接状态。其他任务只需要检查对应的状态位，不需要各自维护网络状态。

Mutex用于保护多个任务可能同时访问的资源，比如共享状态、MQTT客户端、离线缓存和I²C总线，避免数据竞争或多个任务同时操作同一个设备。

11. 为什么共享数据既使用Queue，又使用共享状态？

两者用途不同。

Queue负责传递每一条待上传的数据，强调的是生产者和消费者之间的数据流转。共享状态保存的是温湿度、重量、位置等最新值，OLED只需要显示最近一次结果，不需要按顺序处理所有历史数据。

这样可以避免OLED和上传任务相互等待，也不需要为了显示而消费上传队列中的数据。

四、外设采集与驱动

12. 温湿度和重量数据是如何保证有效性的？

DHT22部分使用成熟库完成协议解析，我在应用层做了读取失败和数据范围检查。如果返回NaN或明显异常值，就不会直接作为有效数据上传，并保留上一组有效值。

重量部分通过HX711读取称重传感器。系统启动时先进行去皮，然后根据已知砝码计算比例系数，采集时通过多次读取取平均值来降低波动。

这属于竞赛原型级处理。如果做成正式产品，还需要增加温漂补偿、长期漂移校准和更严格的异常检测。

13. 你说自己做了外设驱动，是自己从寄存器开始写的吗？

不是全部从寄存器开始写。

DHT22使用了现有DHT库，GPS使用TinyGPSPlus解析NMEA数据，OLED使用U8g2，摄像头使用Espressif提供的 `esp_camera` 组件。我主要完成的是接口配置、任务化封装、数据校验以及这些外设在整个系统中的集成。

因此，更准确的说法是我完成了外设接入、采集逻辑和驱动集成，而不是所有协议都从寄存器层重新实现。

14. GPS为什么从软件串口改成硬件UART？

软件串口依赖软件控制采样时序。当系统同时运行Wi-Fi和多个FreeRTOS任务时，调度延迟可能影响串口接收，导致字符丢失或NMEA语句不完整。

后来我把GPS迁移到ESP32-S3的硬件UART，并把接收缓冲区扩大到2048字节，再通过独立任务持续读取并交给TinyGPSPlus流式解析。

这个改动从原理上降低了CPU负担和数据丢失风险。不过当时没有保留严格的丢包率对比数据，所以我不会声称丢包率具体下降了多少。

五、MQTT与HTTP通信

15. 为什么同时使用MQTT和HTTP？

因为两类数据的特点完全不同。

温湿度、重量、位置和设备状态数据量小、频率低，而且需要设备状态管理和指令下发，所以使用MQTT比较合适。

图像数据量大，而且是连续传输。如果通过MQTT传输，就需要做分片、重组和超时处理，还可能占用消息链路，影响传感器数据和控制命令。因此，图像单独通过HTTP提供JPEG图片或MJPEG流。

本质上，MQTT负责轻量遥测和控制，HTTP负责大数据量的图像传输。

16. MQTT上云是怎么实现的？

我先在阿里云物联网平台创建产品和设备，配置ProductKey、DeviceName和DeviceSecret，然后根据系统需要定义温湿度、重量、经纬度和设备状态等物模型属性。

设备连接阿里云后，把采集数据封装成Alink JSON格式并发布到对应Topic。设备还会订阅控制Topic，用来接收采样周期调整和OTA地址等指令。

程序中还实现了心跳检测和自动重连，避免连接断开后必须人工重启。

17. 为什么MQTT使用QoS 0，数据不会丢吗？

这个项目上传的是周期性遥测数据，下一次采集会继续产生新数据。相比每条消息都必须确认，我们更关注通信开销和实时性，所以在线上传使用QoS 0。

为了解决网络断开时的数据丢失问题，我在设备端增加了离线缓存。网络恢复后再进行限速补发。

这个方案不能保证严格意义上的端到端“零丢失”。如果是计费、告警或安全事件，就需要进一步使用QoS 1、消息编号、服务端确认和持久化存储。

六、OTA、离线缓存与参数持久化

18. 你的OTA是怎么设计的？

我设计了三种升级入口。

第一种是Web OTA，通过设备本地网页上传固件；第二种是局域网OTA，开发人员可以通过ArduinoOTA推送固件；第三种是MQTT下发固件下载地址，设备收到地址后通过HTTP下载并升级。

基本原理是把新固件写入非当前运行的应用分区，写入和校验成功后修改启动分区并重启，从新固件启动。这样不会直接覆盖正在运行的固件。

目前它还是原型方案，没有完整实现HTTPS证书校验、固件签名、安全启动、自动回滚和升级后的健康确认。产品化时必须补上这些机制。

19. 设备断网以后，数据怎么处理？

采集任务不会因为断网停止。数据正常进入上传队列，如果MQTT不可用或者发布失败，就把数据写入RAM环形缓存。

缓存最多保存48条记录。按照默认5秒采集一次计算，大约可以保留4分钟数据。缓存满后覆盖最旧的数据，保证设备仍然能够继续运行。

网络恢复后，系统每轮最多补发4条，并在消息之间留出间隔，避免大量历史数据瞬间占满MQTT链路。这个方案能够处理短时间断网，但设备重启后RAM数据会丢失，也不适合长时间离线。

20. 为什么离线数据放在RAM，而参数放在NVS？

因为两类数据的写入特点不同。

遥测数据产生频率高，如果每隔几秒都写入Flash，会增加擦写次数并影响寿命，所以短时间断网数据放在RAM环形缓存中。

采样周期等配置参数修改频率很低，但需要掉电保存，因此适合写入NVS。设备启动时从NVS读取参数，如果没有有效值就使用默认配置。

简单来说，RAM缓存解决短时断网，NVS解决配置掉电保存。

七、稳定性与优化

21. 你是怎么避免网络阻塞和看门狗复位的？

最初如果把网络连接和重试全部写在主循环中，一旦网络异常，整个系统都会停在重连过程里。

后来我把网络管理拆成独立任务，并用状态和时间间隔控制重连。传感器采集、GPS解析和显示任务不会直接等待网络连接。循环和等待过程中使用FreeRTOS延时主动让出CPU，避免高优先级任务长时间忙等。

严格来说，当前版本并不是完全异步。Wi-Fi连接和MQTT连接内部仍然可能存在有限时间的阻塞，但这些阻塞被限制在网络任务中，不会再阻塞整个系统。

22. 如果Queue满了或者MQTT发送失败怎么办？

如果上传队列满了，或者上传任务取出数据后MQTT发布失败，数据会转入离线环形缓存，避免因为网络暂时拥塞直接丢弃。

不过当前缓存容量有限，缓存满后会覆盖最早的数据，而且普通遥测和告警数据还没有进行优先级区分。产品化时可以增加消息序号、告警优先级和Flash持久化队列。

八、AI联调、测试与项目边界

23. YOLOv8识别是你做的吗？你负责了哪一部分？

YOLOv8模型训练和算法实现不是我负责的，是团队中其他成员完成的。

我负责ESP32-CAM端的图像采集和HTTP输出，为PC端提供JPEG图片或MJPEG视频流，并和算法成员约定图像格式和访问接口，配合打通“摄像头采集—HTTP传输—PC端取流—YOLOv8识别”这条链路。

所以我的贡献是嵌入式图像端和接口联调，不会把模型训练说成是我独立完成的。

24. 项目是怎么测试的？

测试过程是从单模块到完整链路逐步进行的。

首先分别验证温湿度、重量、GPS、OLED和摄像头能否正常工作；然后验证ESP32-S3连接Wi-Fi、登录阿里云以及MQTT数据上报；之后与后端、小程序、数据大屏和YOLOv8模块进行接口联调。

最后，我们在真实蜂场中测试了温湿度和重量采集、物联网通信以及胡蜂识别链路，确认系统能够完成基本的现场采集、传输和展示。

25. 这个项目取得了什么结果？为什么没有量化指标？

最终，我们完成了一个可以实际运行的智慧蜂箱原型，打通了传感器采集、设备上云、后端存储、小程序和数据大屏展示，以及摄像头到YOLOv8识别的完整链路，并在真实蜂场进行了功能测试。项目最后获得了全国大学生物联网设计竞赛全国一等奖。

项目当时的重点是验证完整方案的可行性，没有进行长期稳定性、大规模部署和标准化性能测试，因此没有可靠的在线率、功耗、通信延迟或丢包率数据。我认为没有测试过的数据不应该为了简历效果进行虚构。

26. 这个项目目前有哪些不足？

首先，它还是竞赛原型，硬件在防水、防尘、供电、低功耗和工业环境适应性方面不够完善。

其次，离线缓存使用RAM，只适合短时间断网；HTTP图传主要适用于局域网，远程访问和多客户端能力有限；OTA也缺少签名验证、回滚和升级健康检查。

另外，系统没有进行长期连续运行和多蜂箱并发测试。如果继续产品化，我会优先完善硬件防护和供电，再补充持久化消息队列、安全OTA、设备监控以及长期可靠性测试。

27. OTA、离线缓存这些功能是在比赛时就全部完成的吗？

比赛阶段主要完成了智慧蜂箱的核心功能和整体链路，后续我又围绕简历中的嵌入式方向，对原有项目进行了持续整理和优化，包括FreeRTOS任务架构、OTA、离线缓存和参数持久化等。

这些功能都属于同一个项目的后续技术完善，但如果面试官询问具体时间，我会如实说明哪些是比赛阶段完成的，哪些是比赛结束后继续优化的。

28. 如果让你重新做一次，你会优先改进什么？

我会优先从产品化角度改进三个方面。

第一是硬件可靠性，包括防水防尘、稳定供电、低功耗和传感器长期校准。第二是数据可靠性，把RAM缓存升级为具有消息编号和断电恢复能力的持久化队列。第三是安全和运维能力，包括HTTPS、固件签名、安全启动、OTA回滚以及设备运行状态监控。

在这些基础上，再进行多蜂箱部署、长期运行和功耗等量化测试，让它从竞赛原型逐步变成能够长期使用的产品。

（二）项目行为类高频追问

1. 你在这个项目中遇到的最大困难是什么？

我遇到的最大困难，是如何让传感器采集、GPS解析、本地显示和网络通信这些功能同时稳定运行。

单独测试每个模块时都没有太大问题，但集成到一起以后，网络连接和重连可能占用较长时间，进而影响传感器采集和GPS数据解析。最初如果把这些功能都放在一个大循环里，一个环节发生阻塞，其他功能也会跟着停顿。

后来我使用FreeRTOS把采集、GPS、显示、网络管理和数据上传拆成独立任务，并用Queue传递采集数据、EventGroup管理网络状态、Mutex保护共享资源。这样即使网络发生波动，本地采集和显示也可以继续运行。

这件事让我认识到，嵌入式系统中“每个模块都能工作”并不等于“整个系统能够稳定工作”，系统集成时还必须考虑任务调度、阻塞和资源竞争。

2. 有没有遇到过一个比较具体的问题？你是怎么排查的？

比较典型的是GPS数据接收问题。

GPS单独运行时能够正常解析，但系统同时运行Wi-Fi和其他任务以后，偶尔会出现定位数据更新不及时或者NMEA语句不完整的情况。我当时先把问题分成硬件信号、串口接收和协议解析几个层次进行排查。

最后发现，软件串口比较依赖CPU及时处理，在多任务和网络通信场景下容易受调度延迟影响。因此，我把GPS从软件串口迁移到硬件UART，扩大接收缓冲区，并创建独立任务持续读取串口数据，再交给TinyGPSPlus流式解析。

这个问题让我形成了一个排查习惯：先把完整链路拆开，判断问题发生在采集、传输、解析还是业务处理层，再针对具体层次进行修改。

3. 你是怎么和团队成员进行沟通协作的？

我们按照系统模块进行了职责划分。我主要负责嵌入式终端，其他成员分别负责后端、YOLOv8、小程序和数据大屏。

因为这些模块最终需要连接起来，所以我在开发过程中会重点和其他成员确认接口，例如嵌入式端上传哪些字段、数据采用什么格式、后端订阅什么Topic，以及PC端怎样访问ESP32-CAM提供的图像接口。

联调出现问题时，我们会沿着数据链路逐层检查：先确认嵌入式端是否产生了正确数据，再检查阿里云是否收到、后端是否成功解析，最后检查前端是否正确展示。图像链路也是先验证摄像头能否输出，再检查PC端能否取流，最后验证YOLOv8能否处理。

我在团队协作中的主要作用，就是保证嵌入式端接口明确、输出稳定，并配合其他成员完成上下游联调。

4. 如果联调时发现问题，你怎么判断是谁负责的模块出了问题？

我不会一开始就按照人员划分问题，而是先按照数据链路确定故障位置。

例如小程序没有显示温度数据，我会先检查传感器是否读取成功，然后检查ESP32-S3是否完成JSON封装和MQTT发布，再查看阿里云Topic中是否收到数据。如果云端已经收到，就继续检查后端解析、数据库和前端接口。

这样可以先用日志和实际数据确定问题发生在哪一个环节，再由对应模块的成员处理。相比直接判断“是谁的代码有问题”，这种方式更容易推进联调，也能减少没有依据的相互猜测。

5. 这个项目给你最大的收获是什么？

最大的收获是，我对嵌入式开发的理解从“把外设驱动起来”扩展到了“设计一个完整、可维护的终端系统”。

以前可能更关注传感器能不能读到数据，但这个项目让我开始考虑任务之间如何解耦、网络异常会不会影响本地业务、共享资源如何保护、断网数据如何处理，以及设备部署以后怎样升级。

另外，我也认识到，一个完整物联网项目不可能只依靠单个模块完成。嵌入式、云平台、后端、算法和前端之间必须提前明确接口，并通过逐层联调把完整链路跑通。
